Modelos Bioinspirados y
Heurísticas de Búsqueda
4 Curso Grado Ingeniería en
Informática
Área de Ciencias de la Computación e
Inteligencia Artificial
Departamento de
Tecnologías de la Información

PRÁCTICA 1 (Versión 2026, 1.0)

Algoritmos basados en Entornos y Trayectorias

Objetivos

El objetivo de esta práctica es estudiar el funcionamiento de los *Algoritmos de Búsqueda Aleatoria, Local, Enfriamiento Simulado y Búsqueda Tabú*. Para ello, se requerirá que el alumno implemente estos algoritmos, para resolver un problema de *optimización de balanceo de una red de bicicletas.* El comportamiento de los algoritmos implementados deberá compararse con un *Algoritmo Greedy*.

Problema de la optimización de la red de bicicletas en una ciudad

El problema a resolver consiste en la optimización operativa de las rutas de reubicación de bicicletas de un sistema de préstamo con infraestructuras de **capacidad fija**. Para garantizar el correcto funcionamiento del servicio, se cuenta con un vehículo (camión) de capacidad limitada L, encargado de equilibrar la red mediante un proceso de redistribución periódico.

El algoritmo debe determinar de forma autónoma qué estaciones visitar, en qué orden (ruta) y cuántas bicicletas recoger o depositar en cada una, garantizando que haya tanto bicicletas disponibles para iniciar viajes como anclajes libres para finalizarlos. El proceso de optimización y redistribución se ejecuta al principio de la jornada. La ruta redistribución debe comenzar obligatoriamente en la Estación 0. Conocemos las coordenadas x, y de cada estación, la capacidad actual y máxima. Se pide optimizar **el número de kilómetros recorrido por el camión que reparte las bicicletas y la entropía del sistema** (que es máxima cuando las estaciones están al 50%) con un valor optimo igual al número de estaciones.

Parámetros del Entorno

16 Estaciones y un camión con capacidad de 20 bicicletas que comienza con 7 bicicletas y pueden quedar en él las bicicletas restantes del ultimo movimiento al final de la ruta.

Caso1

Id	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bicis	5	7	13	6	8	13	8	9	6	10	10	18	8	13	15	14
Capacidad	16	16	23	13	21	17	14	13	17	16	27	18	12	18	18	19

Caso 2

Id	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bicis	15	14	3	10	2	1	8	13	17	1	20	9	6	6	15	14
Capacidad	16	16	23	13	21	17	14	13	17	16	27	18	12	18	18	19

Caso 3

Id	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bicis	15	14	3	10	2	1	8	13	10	0	0	0	0	0	0	0
Capacidad	15	14	3	10	2	1	8	13	17	16	27	18	12	18	18	19

Posiciones de las estaciones

Estación	Latitud	Longitud
0	43.461719	-3.8021
1	43.47441267	-3.785781345
2	43.47573226	-3.798063871
3	43.47847181	-3.788653146
4	43.47196611	-3.792854496
5	43.47110611	-3.800586459
6	43.47016611	-3.806499478
7	43.47180483	-3.781346135
8	43.46914941	-3.773275976
9	43.46355212	-3.788005479
10	43.46214308	-3.797241449
11	43.45796625	-3.824801242
12	43.46072554	-3.817978245
13	43.45838389	-3.810468391
14	43.45281305	-3.871390683
15	43.45482606	-3.866900669

El algoritmo debe generar una solución **Ruta de Redistribución (R):** La secuencia ordenada de estaciones que visitará el camión. Ejemplo: R=[1,v1,v2,...,vn]. No es estrictamente necesario visitar todas las estaciones del sistema si su estado de inventario es óptimo. Si la estación está por encima del 50% el camión recogerá el máximo que pueda hasta cumplir ese objetivo sin exceder su capacidad. Asimismo, si está por debajo del 50%, el camión soltara las bicis que pueda para llegar al 50% mientras tenga disponibilidad.

Para determinar si una estación necesita ser balanceada estimareis un limite por encima/debajo del 50% y aunque esté en el vector solución, no se irá a ella y por tanto no entra en el problema , esto se puede hacer a priori antes de empezar el problema y reduciría el numero de posiciones del vector. O se puede hacer durante el calculo de la ruta, y tendremos posiciones que no son visitadas.

Función Objetivo

El propósito del modelo es **minimizar la distancia total generada en el sistema y la entropía al inicio**,

Los kilómetros totales seran varias veces superior a la entropía en un caso pequeño y al revés en un caso con muchas estaciones, por lo que si se opta por sumar ambas cantidades, una solución *mediocre* (porque dos soluciones con una de las partes mala y la otra buena) darian valores similares, no guiando bien al algoritmo, es tener una suma ponderada con un coeficiente *Alpha.*

Fobj = Kms + *Alpha* * Entropia

Para obtener mejores valores estudiar el uso de otras formulas, ratio kms/entropía o exponenciales, etc… proponer la mejor según el resultado.

kilómetros Camión : Representa la suma de los kilómetros recorridos por el camión siguiendo la ruta R establecida por el algoritmo usando la distancia manhatan entre los puntos según el siguiente código (se puede usar otro equivalente) :

```
def distancia_manhattan_km(lat1, lon1, lat2, lon2):
    R = 6371.0 # Radio de la Tierra en km
    dlat_rad = math.radians(abs(lat2 - lat1))
    dlon_rad = math.radians(abs(lon2 - lon1))
    lat_media_rad = math.radians((lat1 + lat2) / 2.0)
    dist_norte_sur = R * dlat_rad
    dist_este_oeste = R * dlon_rad * math.cos(lat_media_rad)

    distancia_total = dist_norte_sur + dist_este_oeste
    return distancia_total

# ejemplo:
lat1, lon1 = 43.461719, -3.8021
lat2, lon2 = 43.47441267, -3.785781345
distancia = distancia_manhattan_km(lat1, lon1, lat2, lon2)

>>>Distancia Manhattan: 2.7284 km
```

- Entropia : dada por la formula :

$$p_i = \frac{B_i}{C_i}$$

$$H = \frac{1}{n}\sum_i -p_i \log(p_i) - (1 - p_i)\log(1 - p_i)$$

Código ejemplo de cálculo de entropía (máximo N)

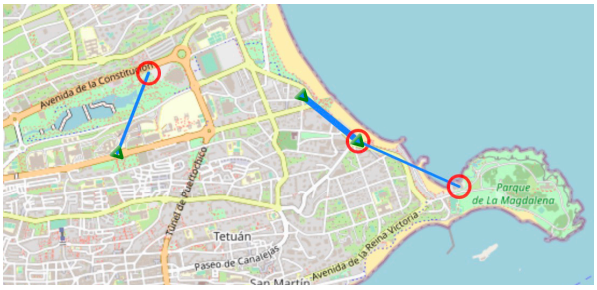
```
def calcular_entropia_total(b, c):

   # b: número de bicicletas en cada estación.
   # c: capacidad máxima de cada estación.
    entropia_total = 0.0
    for bi, ci in zip(b, c):
        # Si la estación está vacía o llena, la entropía es 0 (no sumamos nada)
        if bi == 0 or bi == ci:
            continue
        p = bi / ci
        h_i = -p * math.log2(p) - (1 - p) * math.log2(1 - p)
        entropia_total += h_i

    return entropia_total

c = [20, 15, 30, 25, 10, 20, 40, 15, 20, 10, 25, 30, 15, 20, 10, 25]
b = [10,  0, 30, 20,  5,  2, 35, 10, 10,  8, 12, 15,  5, 18,  1, 12]
entropia = calcular_entropia_total(b, c)

>>>Entropía total del sistema: 11.2287 (Máximo posible: 16.0)
```

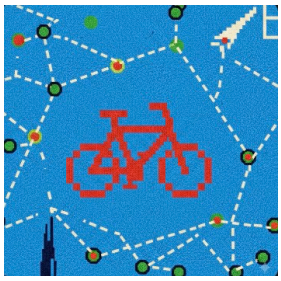
Restricciones del Sistema

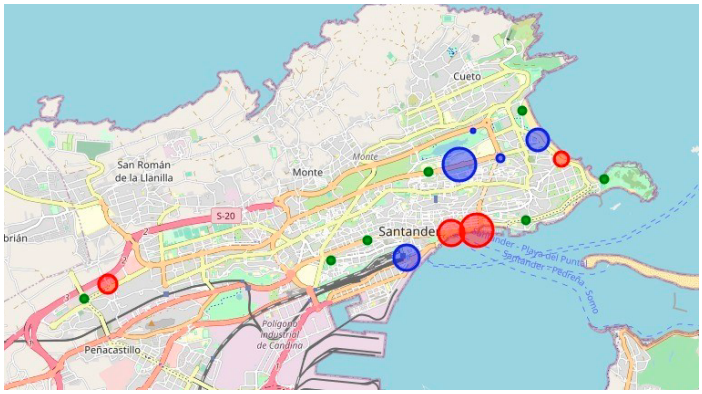
Toda solución generada por el algoritmo debe cumplir:

1. **Inventario de Estaciones:** Tras la intervención del camión, el número de bicicletas resultante en cada estación (Bi + Ri) debe ser mayor o igual a cero, y nunca debe superar la capacidad fija de la estación (Ci). Ri es el numero de bicis que decidimos quitar o reponer en cada estación. i es la estación que visitamos en la posición i de nuestro vector
   - $0 \leq Bi+Ri \leq Ci$
2. **Capacidad del Camión:** En cualquier instante de la ruta, la carga acumulada del camión (Q) no puede ser negativa ni superar su límite máximo. L = 20
   - $0 \leq Q \leq L$
3. **Origen:** La ruta R siempre debe iniciarse en la Estación 0 y acabar de nuevo en la 0. Por lo que el vector a optimizar tendrá n-1 posiciones.

- **Avanzado:**
  - **Genera un trayecto sobre el mapa de santander con el recorrido del camión con las cantidades que suelta y coge de bicis en cada uno en distimto color (verde para ningún cambio o no visitado, azul para dejar y rojo para coger). Este mapa nos permite visualizar como se comporta el recorrido frente a la distancia**
  - **Genera un comparativo del porcentaje de llenado de cada estación con colores , donde no aparecen los trayectos , sólo un circulo de color Azul % por encima de 50%, Rojo % por debajo del 50% y tamaño proporcional Punto Verde si están justos al 50%.**
  - **En Python Folium es una librería de fácil uso.**



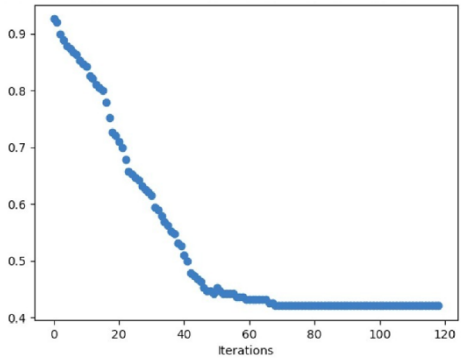




Pseudo-Código Base

El siguiente pseudo-código es total o parcialmente reutilizable en futuras prácticas . Códifica con la suficiente generalización para su uso en los algoritmos de esta y siguientes prácticas con la denominación , parámetros y retorno que estimes oportuno. Ten en cuenta el uso de semillas preestablecidas para el generador de números aleatorios.
Las soluciones deben ser siempre válidas dentro de los mínimos y máximos de capacidad

Ejecución del algoritmo:
  Carga en un vector las 5 semillas
Para cada Semilla:
  Ejecuta el algoritmo con la semilla
    En cada Iteración del algoritmo (cada vez que cambia la solución actual, que no tiene porque coincidir con la mejor solución en simulado y Tabú , guardar iteración y valor Función objetivo en un vector
  Guardar en un array , numero de la ejecución y su mejor valor
  Dibujar gráfica Mejor Valor Objetivo , Valor de Kms de esa solución y Entropia de esa solución por cada iteración . Se aprecia mejor si se dibujan las tres en el mismo gráfico con colores.



El algorimo necesita:
a) Función de evaluación.
b) Generación de la solución inicial.
c) Operador de movimiento . El operador de movimiento debe elegir dos posiciones del vector de estaciones e intercambiar sus posiciones. En Primer Mejor y Mejor Vecoino es llamado con un bucle que va mandando las posiciones i,j

Un ejemplo: partiendo de 0 de 6 estaciones

Camión parte con 7 bicis y tiene 15 de capacidad

3	5	2	4	1

Estación	0	1	2	3	4	5
Bicis	10	10	0	20	15	3
Capacidad	26	20	20	20	20	14

I = esta en 0, deja 3 bicis se queda en 13, Camión 4 bicis
I = 1, va a 3, quita 10 bicis se queda en 10, Camión 14 bicis, Suma Distancia (0,3)
I= 2, va a 5, deja 4 bicis se queda en 7, Camión 9 bicis, Suma Distancia (3,5)
I = 3 ,va a 2, deja 1 9 bicis se queda en 11, Camión 0 bicis ….

Algoritmo de comparación: *Greedy*

Para efectuar la comparativa de resultados entre los distintos algoritmos de búsqueda, se debe implementar como algoritmo básico, un *Greedy*, que partiendo de la primera estación y tras intentar equilibrar esta se viaja a la estación más cercana y se intenta equilibrar hasta que recorre todas y vuelve al principio.

Búsqueda Aleatoria

El Algoritmo de Búsqueda Aleatoria (BA) consistirá en generar aleatoriamente una solución en cada iteración debiéndose ejecutar 100 iteraciones con cada semilla devolviendo la mejor de las iteraciones.

La búsqueda aleatoria completa debe ejecutarse 5 veces, cada vez con una semilla distinta (por tanto, se deben anotar las 5 semillas que se utilizarán sistemáticamente), para el generador aleatorio.

Búsquedas Locales simples: Mejor Vecino y Primer Mejor

Mejor Vecino:

Se recorre todo el entorno de la solución actual , teniendo en cuenta que el primer elemento no se mueve y por tanto se intercambian posiciones desde 2 hasta n.

Se partirá de una solución inicial aleatoria. Los algoritmos de búsqueda local tienen su propia condición de parada, pero adicionalmente, en prevención de tiempos excesivos en algún caso, se añadirá una condición de parada alternativa (OR) basada en el número de evaluaciones que esté realizando la búsqueda, es decir, el número de veces se llame al cálculo de la función de coste. Este valor para la Búsqueda Local será de 3000 llamadas a la función de coste.

La búsqueda con cada algoritmo se debe ejecutar 5 veces con semillas distintas, como en el caso de la Aleatoria.

Primer mejor:

Se implementará saliendo del bucle de elección de vecino cuando se hayan encontrado al menos 3 mejores soluciones o se haya explorado todo el entorno del mismo , según el Tema 1 de teoría.

Debe asegurarse de que, en caso de que no se encuentre hasta 3 mejor vecino , se explore todo el entorno de esta solución. Para otros algoritmos, las dos posiciones se generarán aleatoriamente, por lo que la función base puede tomar dos índices como parámetros para que sea reutilizable.

El código de este algoritmo puede ser similar al anterior , pero realizando una reordenación aleatoria antes de entrar en los dos bucles de generación del entorno, y con un break en el bucle interno que sale si ya se han encontrado los tres vecinos que mejoran a la solución actual.

Enfriamiento Simulado

Se ha de implementar el algoritmo de Enfriamiento Simulado (ES) con las siguientes componentes:

- *Esquema de enfriamiento*: Se implementará el esquema de *Cauchy*,

$$Tk = T0/(1 + k)$$

- *Condición de enfriamiento* $L(T)$: Se enfriará la temperatura, y finalizará a la iteración actual, cuando se haya generado un número máximo de vecinos (independientemente de si han sido o no aceptados). **Determinar este parámetro con la experimentación de al menos tres valores en el entorno de 20 vecinos.**

- *Condición de parada*: El algoritmo finalizará cuando se alcance un número máximo de iteraciones (enfriamientos). **Determinar este parámetro con la experimentación de al menos tres valores en el entorno de 80 iteraciones**.

Se calculará la temperatura inicial en función de la siguiente fórmula:

$$T_0 = \frac{\mu}{-\log(\phi)} \cdot C(S_i)$$

donde $T0$ es la temperatura inicial, $C(Si)$ es el costo de la solución inicial y $\Phi[0,1]$ es la probabilidad de aceptar una solución un μ por 1 peor que la inicial. En las ejecuciones se considerar Φ,μ, entre 0.1 y 0.3 mediante una pequeña experimentación que determine el que el número de soluciones iniciales no aceptadas sea de un 20% para T0, tomando C(s) como el coste de la solución Greedy.

Se debe repetir también 5 veces con distintas semillas, partiendo de una solución inicial aleatoria.

Ejemplo Práctico Paso a Paso

1. Obtén coste base C(Si)

Primero, ejecutas tu algoritmo Greedy para el conjunto de datos que estés evaluando. Imagina que te devuelve una ruta cuya Función Objetivo (Kms + Alfa * Entropía) da un valor de **500**. Entonces, C(Si)=500.

2. Elige valores iniciales para probar

Debes considerar Φ y μ entre 0.1 y 0.3. Vamos a empezar probando con:

- μ=0.15
- Φ=0.20

3. Calcula tu T0 provisional

Aplicamos la fórmula:

$T0=-\ln(0.20)0.15\cdot 500$
$T0=1.6090.15\cdot 500\approx 46.6$

Tu temperatura inicial de prueba es **46.6**.

4. Realiza la "pequeña experimentación"

Ahora necesitas comprobar si esta temperatura de 46.6 cumple la regla del 20% de rechazo. Para ello, escribes un pequeño bucle temporal en tu código:

- Fijas la temperatura actual $T=46.6$.
- Generas, por ejemplo, **100 vecinos aleatorios** a partir de tu solución inicial.
- Para cada vecino, calculas su diferencia de coste (Δ).
- Decides si lo aceptas o lo rechazas usando la lógica del Enfriamiento Simulado:
  - Si es mejor ($\Delta<0$), se acepta.
  - Si es peor ($\Delta>0$), se acepta si $\text{random}(0,1) < e^{-\Delta/T}$.
- **Cuentas cuántos vecinos han sido rechazados.**

5. Evalúa y ajusta

Aquí es donde determinas si tus valores de μ y Φ fueron correctos o si debes cambiarlos:

- **Caso A (Rechazas demasiado):** Imagina que has rechazado 45 vecinos (45%). Esto significa que tu temperatura está muy fría y te cuesta aceptar soluciones peores. Debes **aumentar** la temperatura. Para ello, repites el paso 2 subiendo μ (ej. a 0.25) o subiendo Φ (ej. a 0.25).
- **Caso B (Rechazas muy poco):** Imagina que has rechazado solo 2 vecinos (2%). Tu temperatura está ardiendo y aceptas casi todo. Debes **bajar** la temperatura. Para ello, repites el paso 2 bajando μ o bajando Φ.
- **Caso C (El caso ideal):** Si has rechazado alrededor de 20 vecinos (~20%), Has encontrado que $\mu=0.15$ y $\Phi=0.20$ funcionan perfectamente para este dataset. Te guardas ese $T0=46.6$ y, lanzas tu algoritmo de Enfriamiento Simulado real.

Para las experimentaciones previas se puede tomar un conjunto de datos reducido siempre que los resultados sean extrapolables.

Búsqueda Tabú

Se implementará la versión de BT utilizando una lista de movimientos tabú y tres estrategias de reinicialización.

Sus principales características son:

- Estrategia de selección de vecino: examinar 20 vecinos para coger el mejor de acuerdo a los criterios tabú.
- Codificación: Una matriz bidimensional de enteros Frecuencia[N][N] (donde N es el número de estaciones/nodos).
- Actualización: Al final de cada iteración, si tu ruta actual viaja del nodo i al nodo j, incrementas el valor: Frecuencia[i][j] += 1.
- Lista Tabú: se admitirá cualquier propuesta con suficiente capacidad de restricción de los movimientos. Una opción válida para la codificación tabú puede ser tener en cuenta sólo que los movimientos realizados no sean los mismos.
  Si partiendo del vector [1, 3, 2, 4] , cambiamos posición 2 y 3 --> [1, 2, 3, 4] , la codificación tabú por movimientos --> [2, 3] cualquier vecino que no incluya cambios de uno de esos indices o cumpla el criterio de aspiración sería un vecino válido.
- Selección de estrategias de reinicialización: La probabilidad de escoger la reinicialización construyendo una solución inicial aleatoria es 0,25, la de usar la memoria a largo plazo al generar una nueva solución greedy es 0,5, y la de utilizar la reinicialización desde la mejor solución obtenida es 0,25.
- La solución greedy debe generar soluciones con mayor probabilidad para los valores que menos veces se han producido en cada estación pero teniendo en cuenta sólo las 5 menos frecuentes para evitar el efecto aleatorio de muchos nodos.

Para cada estación se deberá calcular las inversas de los valores acumulados y luego normalizar para tener valores entre 0-1 correspondiente a su probabilidad. Se tirará un dado y se elige el primer valor que supera dicho valor añadiéndose a la solución greedy

*Atención: usar una resolución alta de decimales o tener en cuenta la posibilidad de que el algoritmo pueda salir sin encontrar el último valor.

Una vez hecho esto todas las posiciones de la matriz *matrizProbabilidades* tienen su probabilidad directa y se pasa a construir un greedy con a las proporciones de esta matriz y las distancias entre estaciones.

```
Algoritmo Generico
        soluciónGreedy = {}
        Para cada estación en estaciones
            Número = random(0,1)
            suma=0
            Para i = valor en valores
                Suma+= matrizProbabilidades (estación, i)
                If número < suma
                  soluciónGreedy.add(valor)
                    break()

        return soluciónGreedy
```

Proceso Lógico: Selección por Ruleta Inversa de Frecuencias

Paso 1: Identificar y ordenar candidatas

- Estando en la `estación_actual`, mirar en la matriz de frecuencias cuántas veces se ha viajado hacia cada una de las estaciones **no visitadas**.
- Ordenar estas estaciones candidatas de **menor a mayor frecuencia**.

Paso 2: Crear la Lista Restringida (Top 5)

- Seleccionar únicamente las **5 estaciones con menor frecuencia** (o menos, si quedan menos de 5 estaciones por visitar). Estas serán nuestras opciones viables.

Paso 3: Calcular el "Peso Inverso"

- Para que las estaciones con *menor* frecuencia tengan *mayor* probabilidad de salir elegidas, calculamos un peso inverso para cada una de las 5 candidatas.
- *Ojo!:* Como la frecuencia puede ser 0 (nunca visitada), no podemos dividir directamente entre 0. Se debe sumar 1 al denominador.
- Fórmula del peso para una estación i:

  Wi=Frecuenciai+11

Paso 4: Calcular las probabilidades (La Ruleta)

- Sumar todos los pesos de las 5 estaciones para obtener un `Peso_Total`.
- La probabilidad de elegir una estación i será su peso dividido entre el total:

  Probi=Peso_TotalWi

Paso 5: Tirar el dado

- Generar un número aleatorio entre 0 y 1 (o entre 0 y 100%).
- Recorrer las 5 estaciones sumando sus probabilidades de forma acumulada hasta que la suma supere el número aleatorio. La estación donde esto ocurra es la elegida.

Ejemplo Numérico para la Clase

Imaginemos que estamos en la **Estación 0**. Quedan varias estaciones por visitar, y miramos nuestra matriz de frecuencias para ver cuántas veces hemos ido desde la 0 a las demás:

- Estación 1: Frecuencia 4
- Estación 2: Frecuencia 0
- Estación 3: Frecuencia 2
- Estación 4: Frecuencia 1
- Estación 5: Frecuencia 5
- Estación 6: Frecuencia 8

1. Ordenamos de menor a mayor frecuencia: 2 (F=0), 4 (F=1), 3 (F=2), 1 (F=4), 5 (F=5), 6 (F=8).

2. Nos quedamos con el Top 5: Las estaciones seleccionadas son: **2, 4, 3, 1 y 5**. (Descartamos la 6 porque es la sexta opción).

3. Calculamos el Peso Inverso (W=F+11):

- Estación 2: 1/(0+1)=1.00
- Estación 4: 1/(1+1)=0.50
- Estación 3: 1/(2+1)=0.33
- Estación 1: 1/(4+1)=0.20
- Estación 5: 1/(5+1)=0.16

4. Sumamos y sacamos el porcentaje (Peso Total = 2.19):

- Estación 2: 1.00/2.19=45.6% de probabilidad.
- Estación 4: 0.50/2.19=22.8% de probabilidad.
- Estación 3: 0.33/2.19=15.0% de probabilidad.
- Estación 1: 0.20/2.19=9.1% de probabilidad.
- Estación 5: 0.16/2.19=7.3% de probabilidad.

5. Tiramos el dado: Supongamos que el ordenador genera el número aleatorio **61.2%**.

- Revisamos la Estación 2: Cubre del 0% al 45.6%. (No llega).
- Sumamos la Estación 4: 45.6%+22.8%=68.4%. Cubre del 45.6% al 68.4%.
- El 61.2% cae dentro de la porción de la estación 4. El algoritmo elige viajar a la estación 4.

De esta forma te aseguras de que el algoritmo obligue a explorar caminos nuevos de forma semialeatoria, dando siempre mucha más prioridad a lo inexplorado (45.6%) pero dejando la puerta abierta a lo ya conocido (7.3%) para evitar bloqueos.

Estimar el número máximo de iteraciones en total. Se realizarán 4 reinicializaciones, es decir, una cada Numtotal-iteraciones/4. El tamaño inicial de cada lista tabú será n=4, estos valores cambiarán después de las reinicializaciones según se ha comentado.
Además de saltar a una solución concreta según las reinicializaciones comentadas, también se alterará un parámetro del algoritmo de búsqueda para provocar un cambio de comportamiento del algoritmo más efectivo. Este consistirá en variar el tamaño de la lista tabú, incrementándola o reduciéndola en un 50% según una decisión aleatoria uniforme, empezando la lista desde vacío en cada reinicialización.

Dado el carácter probabilístico del algoritmo, deberá ejecutarse 5 veces (con semillas distintas para el generador aleatorio, para cada caso del problema (*dataset*).

Metodología de Comparación

El alumno tendrá que contabilizar el número de evaluaciones (llamadas realizadas a la función de coste) producidas por los distintos algoritmos, que será empleado como una medida adicional de comparación de su calidad.

A partir de la experimentación efectuada, se construirá una tabla global de resultados con la estructura mostrada en la Tabla mostrada. La columna etiquetada con *Mej* indica

el valor de la mejor solución encontrada, la columna σ refiere a la desviación típica y la columna etiquetada con *Ev y Ev. Mejor* indica respectivamente el número medio y mínimo de evaluaciones realizadas por el algoritmo en las cinco ejecuciones (salvo en el caso del algoritmo *Greedy* que sólo se ejecuta una vez).

Ojo: Mejor kms y mejor Entropia es el valor de estas dos cantidades para la mejor solución de la Funcion Objetivo diseñada, no el mejor valor de cada uno en las ejecuciones.

Algoritmo	**Ev.Medias**	**Ev. Mejor**	**σ EV**	**Mejor Kms**	**Mejor Entropia**	**F. Objetivo**
Greedy						
Aleatoria						
BL Primer						
BL Mejor						
Enf. Simulado						
Tabú						

A partir de los datos mostrados en estas tablas y gráficos, el alumno se deben comparar, para cada instancia, las distintas variantes de los algoritmos en términos de: número de evaluaciones, mejor resultado individual obtenido y mejor resultado medio (robustez del algoritmo).

Las prácticas se realizarán individualmente.

Fecha y Método de Entrega;

El día 10 de Abril durante la sesión de prácticas. Debe entregar 1 fichero comprimido ZIP, que contenga:

- Documento DOC (MS Word) ó PDF con las tablas de resultados y análisis.
- Ficheros de código fuente completo ejecutable utilizado.
- Scripts, si los ha utilizado.
- En esta fecha se realiará un examen de modificación sobre la práctica y una defensa oral de los resultados . En caso de no superar por no cumplir los minimos o no defender adecuadamente tendrá otra oportunidad de reentrega (si es necesario corregir la práctica) y presentación a examen de modificación en segunda convocatoria.

La evaluación tendrá en cuenta la siguiente rúbrica:

Nivel Básico (4):

- Corrección del algoritmo
- Utilización de la terminología apropiada (la utilizada en el material de clase)

Nivel intermedio(5-6):

- Análisis e interpretación sobre la capacidad de exploración/explotación de cada algoritmo relacionado con los resultados obtenidos.
- Gráficas y ejemplos concretos comparados.

Nivel Alto (7-8)

- Mejoras en el rendimiento de la aplicación
- Resultados obtenidos razonablemente buenos
- Análisis de la estadística de los resultados. Algoritmos más estables. Desviación típica.

Nivel Avanzado(9-10):

- Graficos de estado sobre el mapa .
- Función de fitness mejorada
- Solución final dibujada